

LAPORAN PRAKTIKUM
ALGORITMA DAN STRUKTUR DATA
PENERAPAN FUNCTIONS DAN ARRAY



DISUSUN OLEH :

NAMA	: Dewi Almas
NIM	: 2025573010009
KELAS	: TI 1D
JURUSAN	: Teknologi Informasi Dan Komputer
PRODI	: Teknik Informatika
DOSEN PENGAMPU	: Muhammad Reza Zulman, S.ST.,M.SC

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
TAHUN 2026

KATA PENGANTAR

Segala puji dan syukur penulis ucapkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga saya dapat menyelesaikan laporan praktikum ini tepat waktu.

Laporan ini dibuat untuk memenuhi tugas mata kuliah Algoritma dan Struktur Data yang membahas tentang penggunaan *function* dan *array* dalam pemrograman, serta praktik pembuatan folder dan pengelolaan file menggunakan Git Bash. Melalui praktikum ini, saya memperoleh pemahaman mengenai bagaimana fungsi digunakan untuk menyederhanakan kode serta bagaimana array digunakan untuk mengelola data secara lebih terstruktur.

Saya menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang bersifat membangun guna meningkatkan kualitas penulisan di masa yang akan datang.

Semoga laporan ini dapat bermanfaat saya sendiri maupun pembaca.

Buket Rata, 14 April 2026

Dewi Almas

Nim.2025573010009

DAFTAR ISI

HALAMAN JUDUL	i
KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Praktikum	1
BAB II LANDASAN TEORI	
2.1 Function (Fungsi)	2
2.2 Array	2
2.3 Rekursi	2
BAB III PEMBAHASAN	
3.1 Mengakses Repository	3
3.2 Pembuatan Folder Praktikum	3
3.3 Pembuatan File Praktikum	4
3.4 Pembuatan File 01-functional-dasar.js	4
3.5 Pembuatan File 02-arrow-callback.js	5
3.6 Pembuatan File 03-array-dasar.js.....	6
3.7 Pembuatan File 04-array-method.js	7
3.8 Pembuatan File 05-rekursi.js	8
3.9 Implementasi Function Dasar (01-function-dasar.js)	9
3.10 Implementasi Arrow Function dan Callback (02-arrow-callback.js)	10
3.11 Implementasi Array Dasar (03-array-dasar.js)	11
3.12 Implementasi Method Array (04-array-method.js)	11
3.13 Pembuatan Folder Tugas	12
3.14 Implementasi Tugas 1 : Sistem Nilai Mahasiswa	13

3.15 Implementasi Tugas 2 : Pangkat dan Palindrom rekursif	14
3.16 Langkah-Langkah Submit Tugas Ke GitHub	15

BAB VI ANALISIS

4.1 Analisis Function Dasar	18
4.2 Analisis Arrow Function	18
4.3 Analisis Callback	18
4.4 Analisis Array Dasar	18
4.5 Analisis Method Array	18
4.6 Analisis Rekursi	19

BAB V PENUTUP

5.1 Kesimpulan	19
5.2 Saran	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut setiap individu untuk memiliki kemampuan dalam bidang pemrograman. Dalam pemrograman, terdapat berbagai konsep dasar yang harus dipahami, di antaranya adalah *function* dan *array*. *Function* digunakan untuk mempermudah penulisan program dengan cara membagi kode menjadi bagian-bagian yang lebih terstruktur dan dapat digunakan kembali. Sedangkan *array* digunakan untuk menyimpan sekumpulan data dalam satu variabel sehingga lebih efisien dalam pengelolaan data.

Selain memahami konsep dasar pemrograman, mahasiswa juga dituntut untuk mengenal penggunaan tools pendukung seperti Git Bash. Git Bash digunakan untuk mengelola file dan folder serta membantu dalam proses version control, sehingga memudahkan dalam penyimpanan dan pengelolaan proyek.

Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami dan mengimplementasikan penggunaan *function* dan *array*, serta mampu menggunakan Git Bash dalam membuat dan mengelola file secara efektif.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, maka rumusan masalah dalam praktikum ini adalah:

1. Apa yang dimaksud dengan *function* dalam pemrograman?
2. Apa yang dimaksud dengan *array* dan bagaimana penggunaannya?
3. Bagaimana cara membuat dan mengelola folder serta file menggunakan Git Bash?

1.3 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah:

1. Untuk memahami konsep dan penggunaan *function* dalam pemrograman.
2. Untuk memahami konsep dan penggunaan *array*.
3. Untuk mengetahui cara menggunakan Git Bash dalam membuat folder dan file.
4. Untuk meningkatkan keterampilan dalam pengelolaan file menggunakan Git.

BAB II

LANDASAN TEORI

2.1 Function (Fungsi)

Function atau fungsi adalah sekumpulan perintah dalam program yang digunakan untuk menjalankan tugas tertentu. Fungsi bertujuan untuk membuat kode menjadi lebih terstruktur, mudah dipahami, dan dapat digunakan kembali.

Dalam praktikum ini, penggunaan function dapat dilihat pada file 01-function-dasar.js, yang berisi contoh dasar pembuatan dan pemanggilan fungsi. Selain itu, pada file 02-arrow-callback.js, digunakan konsep *arrow function* yang merupakan penulisan fungsi yang lebih singkat dan modern dalam JavaScript.

Dengan adanya function, program tidak perlu menuliskan kode yang sama berulang kali sehingga lebih efisien.

2.2 Array

Array adalah struktur data yang digunakan untuk menyimpan banyak nilai dalam satu variabel. Setiap data dalam array memiliki indeks yang dimulai dari 0.

Pada praktikum ini, penggunaan array terdapat pada file:

- 1 03-array-dasar.js → membahas dasar penggunaan array
- 2 04-array-method.js → membahas method atau fungsi bawaan array seperti menambah, menghapus, dan mengolah data

Array sangat membantu dalam pengolahan data yang banyak agar lebih rapi dan mudah diakses.

2.3 Rekursi

Rekursi adalah teknik dalam pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan suatu masalah.

Konsep ini digunakan dalam file 05-rekursi.js, yang bertujuan untuk menyelesaikan permasalahan secara bertahap hingga mencapai kondisi tertentu (base case). Rekursi biasanya digunakan dalam perhitungan matematika atau pengolahan data yang berulang.

BAB III

PEMBAHASAN

3.1 Mengakses Repository

Pada praktikum ini, langkah awal yang dilakukan adalah membuka repository yang telah dibuat atau di-clone pada praktikum sebelumnya. Proses ini dilakukan dengan menggunakan perintah `cd` pada Git Bash untuk masuk ke direktori repository yang sudah ada.

Adapun langkah yang dilakukan adalah berpindah ke drive yang digunakan, kemudian masuk ke folder hingga ke dalam repository utama. Hal ini bertujuan agar seluruh proses pembuatan file dan folder dilakukan di dalam repository tersebut.

```
MINGW64:/d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3
acer@DESKTOP-NJEJB3T MINGW64 ~
$ cd /d
acer@DESKTOP-NJEJB3T MINGW64 /d
$ cd "/d/Struktur data dan algoritma"
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma
$ cd 2025573010009_strukturdatadanalgoritma
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma (main)
```

3.2 Pembuatan Folder Praktikum

Setelah berhasil mengakses repository, langkah selanjutnya adalah membuat struktur folder yang akan digunakan dalam praktikum. Pembuatan folder dilakukan menggunakan perintah `mkdir` pada Git Bash.

Folder yang dibuat terdiri dari folder praktikum-3 dan tugas. Folder praktikum-3 digunakan untuk menyimpan file materi praktikum, sedangkan folder tugas digunakan untuk menyimpan file latihan.

1. Perintah yang digunakan untuk membuat folder praktikum-3: `mkdir praktikum-3`

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma (main)
$ mkdir praktikum-3
```

2. Perintah yang digunakan untuk memanggil folder praktikum-3: `cd praktikum-3`

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma (main)
$ cd praktikum-3
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
```

3. Inisialisasi npm project. Perintah ini akan membuat file `package.json` secara otomatis.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
$ npm init -y
```

4. verifikasi bahwa file package.json sudah berhasil dibuat dengan menulis kalimat” *ls*” di Git Bash.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
$ ls
```

5. Package.json berhasil dibuat

```
a/praktikum-3 (main)
package.json
```

3.3 Pembuatan File Praktikum

Setelah berhasil membuat folder praktikum-3, selanjutnya lakukan pembuatan beberapa file JavaScript lainnya di dalam folder praktikum-3 menggunakan perintah touch.

3.4 Pembuatan File 01-functional-dasar.js

1. Di dalam folder praktikum-3/, buat file baru bernama 01 function-dasar.js dengan mengetik “*touch 01-function-dasar.js*” di Git Bash.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
$ touch 01-function-dasar.js
```

2. Buka file 01 function-dasar.js di VS Code, lalu ketikkan kode berikut.

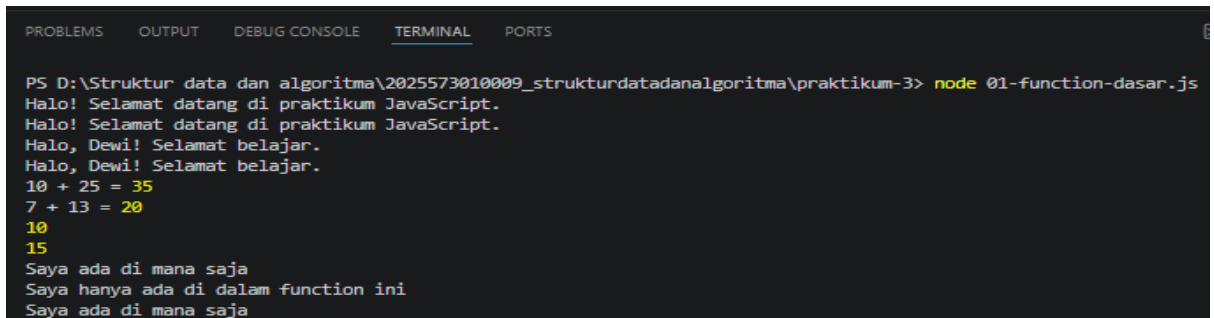
```
JS 01-function-dasar.js X
JS 01-function-dasar.js > ...
1  function sapa() {
2  console.log('Halo! Selamat datang di praktikum JavaScript.');
```

```
3  }
4  sapa();
5  sapa();
6
7  function sapaNama(nama) {
8  console.log('Halo, ${nama}! Selamat belajar.');
```

```
9  }
10 sapaNama('Dewi');
11 sapaNama('Dewi');
```

```
12 function tambah(angka1, angka2) {
13 const hasil = angka1 + angka2;
14 return hasil;
15 }
16
17 const hasilPenjumlahan = tambah(10, 25);
18 console.log('10 + 25 =', hasilPenjumlahan);
19 console.log('7 + 13 =', tambah(7, 13));
20
21 function hitung(nilai, pengali = 2) {
22 return nilai * pengali;
23 }
24 console.log(hitung(5));
25 console.log(hitung(5, 3));
26
27 const pesanGlobal = 'Saya ada di mana saja';
28
29 function cekScope() {
30 const pesanLokal = 'Saya hanya ada di dalam function ini';
31 console.log(pesanGlobal);
32 console.log(pesanLokal);
33 }
34 cekScope();
35 console.log(pesanGlobal);
```

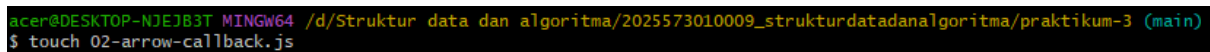

3. Simpan file kemudian jalankan file tersebut dari terminal dengan mengetik “*node 01-function-dasar.js*” di terminal.



```
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-3> node 01-function-dasar.js
Halo! Selamat datang di praktikum JavaScript.
Halo! Selamat datang di praktikum JavaScript.
Halo, Dewi! Selamat belajar.
Halo, Dewi! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada di dalam function ini
Saya ada di mana saja
```

3.5 Pembuatan File 02-arrow-callback.js

1. Buat file baru bernama 02-arrow-callback.js di dalam folder praktikum-3/ dengan mengetik” *02-arrow-callback.js*” di Git Bash.



```
acer@DESKTOP-NJEJ83T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
$ touch 02-arrow-callback.js
```

2. Buka file 02-arrow-callback.js di VS Code, lalu ketikkan kode berikut.



```
JS 02-arrow-callback.js X
JS 02-arrow-callback.js > ...
1  function kuadratBiasa(x) {
2      return x * x;
3  }
4
5  const kuadrat = (x) => {
6      return x * x;
7  };
8
9  const kuadratRingkas = x => x * x;
10 console.log('=== Perbandingan Penulisan ===');
11 console.log('Biasa :', kuadratBiasa(5));
12 console.log('Arrow :', kuadrat(5));
13 console.log('Ringkas :', kuadratRingkas(5));
14
15 const luas = (panjang, lebar) => panjang * lebar;
16 const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;
17 console.log('Luas 4x6 :', luas(4, 6));
18 console.log(salam('Budi', 'Pagi'));
19
20 function lakukanOperasi(angka, operasiCallback) {
21     console.log('Angka awal: ${angka}');
22     const hasil = operasiCallback(angka);
23     console.log('Hasil setelah operasi: ${hasil}');
24 }
25 console.log('\n=== Callback ===');
26
27 lakukanOperasi(7, kuadratRingkas);
28 lakukanOperasi(10, x => x + 100);
29 lakukanOperasi(20, function(x) {
30     return x / 2;
31 });
32 console.log('\n=== setTimeout (Callback) ===');
33 console.log('Pesan 1: Sebelum timer');
34 setTimeout(() => {
35     console.log('Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)');
36 }, 1000);
37 console.log('Pesan 2: Setelah mendaftarkan timer');
```

3. Simpan file, kemudian jalankan dari terminal dengan mengetik “*node 02-arrow-callback.js*” di terminal

```

PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-3> node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
  
```

3.6 Pembuatan File 03-array-dasar.js

1. Buat file baru bernama 03-array-dasar.js di dalam folder praktikum-3/ dengan mengetik” *touch 03-array-dasar.js* ” di Git Bash.

```

acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
$ touch 03-array-dasar.js
  
```

2. Buka file 03-array-dasar.js di VS Code, lalu ketikkan kode berikut.

```

JS 03-array-dasar.js X
JS 03-array-dasar.js > ...
1  const mahasiswa = ['Budi', 'Siti', 'Ahmad', 'Rina'];
2  const nilai = [85, 92, 78, 95, 88];
3
4  console.log('=== Array Awal ===');
5  console.log('Mahasiswa:', mahasiswa);
6  console.log('Nilai :', nilai);
7  console.log('Jumlah mahasiswa:', mahasiswa.length);
8
9  console.log('\n=== Akses Elemen ===');
10 console.log('Elemen pertama :', mahasiswa[0]);
11 console.log('Elemen ketiga :', mahasiswa[2]);
12 console.log('Elemen terakhir:', mahasiswa[mahasiswa.length - 1]);
13
14 mahasiswa[1] = 'Siti Rahayu';
15 console.log('\nSetelah diubah:', mahasiswa);
16
17 console.log('\n=== Manipulasi Array ===');
18 mahasiswa.push('Doni');
19 console.log('Setelah push :', mahasiswa);
20
21 mahasiswa.unshift('Zahra');
22 console.log('Setelah unshift :', mahasiswa);
23
24 const dihapusAkhir = mahasiswa.pop();
25 console.log('Dihapus (pop) :', dihapusAkhir);
26 console.log('Setelah pop :', mahasiswa);
27
28 const dihapusAwal = mahasiswa.shift();
29 console.log('Dihapus (shift) :', dihapusAwal);
30 console.log('Setelah shift :', mahasiswa);
31
32 console.log('\n=== Pencarian ===');
33 console.log('Indeks Ahmad :', mahasiswa.indexOf('Ahmad'));
34 console.log('Ada Rina? :', mahasiswa.includes('Rina'));
35 console.log('Ada Budi? :', mahasiswa.includes('Budi'));
36
37 const sebagian = nilai.slice(1, 4);
38 console.log('\nNilai asli :', nilai);
39 console.log('Slice [1,4] :', sebagian);
  
```

3. Simpan file, kemudian jalankan dari terminal dengan mengetik “*node 03-array-dasar.js*” di terminal

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-3> node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

Nilai asli : [ 85, 92, 78, 95, 88 ]
Slice [1,4] : [ 92, 78, 95 ]

```

3.7 Pembuatan File 04-array-method.js

1. Buat file baru bernama 04-array-method.js di dalam folder praktikum-3/ dengan mengetik” *touch 04-array-method.js*” di Git Bash.

```

acer@DESKTOP-NJEJ83T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
$ touch 04-array-method.js

```

2. Buka file 04- array-method.js di VS Code, lalu ketikkan kode berikut .

```

JS 04-array-method.js X
JS 04-array-method.js > ...
1  const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];
2
3  console.log('=== forEach: Tampilkan Semua Nilai ===');
4  nilaiMahasiswa.forEach((nilai, indeks) => {
5    console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
6  });
7
8  const gradeHuruf = nilaiMahasiswa.map(nilai => {
9    if (nilai >= 90) return 'A';
10   if (nilai >= 80) return 'B';
11   if (nilai >= 70) return 'C';
12   if (nilai >= 60) return 'D';
13   return 'E';
14 });
15
16 console.log('\n=== map: Nilai ke Grade ===');
17 console.log('Nilai :', nilaiMahasiswa);
18 console.log('Grade :', gradeHuruf);
19
20 const nilaiLulus = nilaiMahasiswa.filter(nilai => nilai >= 60);
21 const nilaiGagal = nilaiMahasiswa.filter(nilai => nilai < 60);
22
23 console.log('\n=== filter: Lulus dan Tidak Lulus ===');
24 console.log('Semua nilai :', nilaiMahasiswa);
25 console.log('Nilai lulus :', nilaiLulus);
26 console.log('Nilai gagal :', nilaiGagal);
27
28 const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
29   return akumulator + nilai;
30 }, 0);
31
32 const rataRata = totalNilai / nilaiMahasiswa.length;
33
34 console.log('\n=== reduce: Statistik Nilai ===');
35 console.log('Total nilai :', totalNilai);
36 console.log('Rata-rata :', rataRata.toFixed(2));
37
38 const rataRataNilaiLulus = nilaiMahasiswa
39   .filter(nilai => nilai >= 60)
40   .reduce((acc, val, idx, arr) => {
41     return acc + val / arr.length;
42   }, 0);
43
44 console.log('\n=== Method Chaining ===');
45 console.log('Rata-rata nilai lulus:', rataRataNilaiLulus.toFixed(2));

```

3. Simpan file, kemudian jalankan dari terminal dengan mengetik “*node 04- array-method.js*” di terminal

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanaalgoritma\praktikum-3> node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
Mahasiswa-1: 75
Mahasiswa-2: 90
Mahasiswa-3: 55
Mahasiswa-4: 82
Mahasiswa-5: 68
Mahasiswa-6: 95
Mahasiswa-7: 48
Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]

=== reduce: Statistik Nilai ===
Total nilai : 590
Rata-rata : 73.75

=== Method Chaining ===
Rata-rata nilai lulus: 81.17

```

3.8 Pembuatan File 05-rekursi.js

1. Buat file baru bernama 05-rekursi.js di dalam folder praktikum-3/ dengan mengetik” *touch 05-rekursi.js*” di Git Bash.

```

acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanaalgoritma/praktikum-3 (main)
$ touch 05-rekursi.js

```

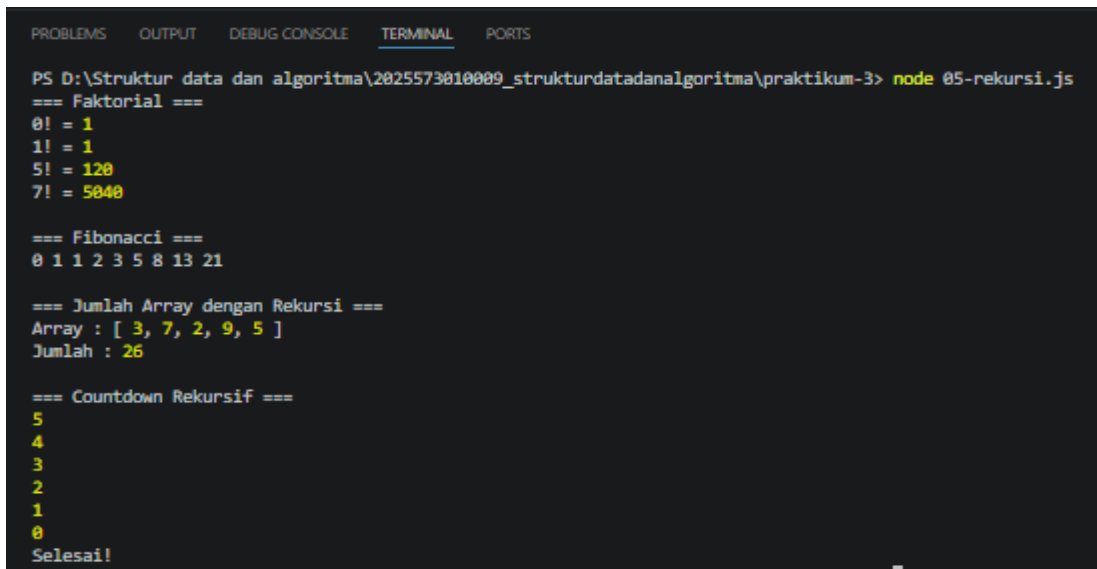
2. Buka file 05-rekursi.js di VS Code, lalu ketikkan kode berikut

```

1 function faktorial(n) {
2   if (n <= 1) return 1;
3   return n * faktorial(n - 1);
4 }
5
6 console.log('--- Faktorial ---');
7 console.log('0! =', faktorial(0));
8 console.log('1! =', faktorial(1));
9 console.log('5! =', faktorial(5));
10 console.log('7! =', faktorial(7));
11
12 function fibonacci(n) {
13   if (n === 0) return 0;
14   if (n === 1) return 1;
15
16   return fibonacci(n - 1) + fibonacci(n - 2);
17 }
18 console.log('\n--- Fibonacci ---');
19 for (let i = 0; i <= 8; i++) {
20   process.stdout.write(fibonacci(i) + ' ');
21 }
22 console.log('');
23
24 function jumlahArray(arr, indeks = 0) {
25   if (indeks === arr.length) return 0;
26   return arr[indeks] + jumlahArray(arr, indeks + 1);
27 }
28
29 const angka = [3, 7, 2, 9, 5];
30 console.log('\n--- Jumlah Array dengan Rekursi ---');
31 console.log('Array :', angka);
32 console.log('Jumlah :', jumlahArray(angka));
33
34 function countdown(n) {
35   if (n < 0) {
36     console.log('Selesai!');
37     return;
38   }
39   console.log(n);
40   countdown(n - 1);
41 }
42
43 console.log('\n--- Countdown Rekursif ---');
44 countdown(5);

```

3. Simpan file, kemudian jalankan dari terminal dengan mengetik “*node 05-rekursi.js*” di terminal



```
PS D:\Struktur data dan algoritma\2025573010009_strukturdadanalgoritma\praktikum-3> node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
```

3.9 Implementasi Function Dasar (01-function-dasar.js)

Pada bagian ini dilakukan implementasi function dasar sesuai dengan modul praktikum. Function digunakan untuk menjalankan suatu perintah tertentu agar program lebih terstruktur.

1. Penulisan kode Latihan Function Dasar



```
7 console.log("\n--- LATIHAN 1 ---");
8
9 function kurang(a, b) {
10   return a - b;
11 }
12
13 function kali(a, b) {
14   return a * b;
15 }
16
17 function bagi(a, b) {
18   if (b === 0) {
19     console.log('Error: tidak bisa membagi dengan nol');
20     return null;
21   }
22   return a / b;
23 }
24
25 function tambah(a, b) {
26   return a + b;
27 }
28
29 function kalkulator(a, operator, b) {
30   let hasil;
31   switch (operator) {
32     case '+':
33       hasil = tambah(a, b);
34       break;
35     case '-':
36       hasil = kurang(a, b);
37       break;
38     case '*':
39       hasil = kali(a, b);
40       break;
41     case '/':
42       hasil = bagi(a, b);
43       break;
44     default:
45       return "Operator tidak dikenal";
46   }
47   return hasil;
48 }
49
50 console.log("--- Uji Coba Kalkulator ---");
51
52 console.log(`10 + 5 = ${kalkulator(10, '+', 5)}`);
53 console.log(`10 - 5 = ${kalkulator(10, '-', 5)}`);
54 console.log(`10 * 5 = ${kalkulator(10, '*', 5)}`);
55 console.log(`10 / 2 = ${kalkulator(10, '/', 2)}`);
56 console.log(`10 / 0 = ${kalkulator(10, '/', 0)}`);
57
```

2. Jalankan hasil output di Terminal

```
=== LATIHAN 1 ===  
--- Uji Coba Kalkulator ---  
10 + 5 = 15  
10 - 5 = 5  
10 * 5 = 50  
10 / 2 = 5  
Error: tidak bisa membagi dengan nol  
10 / 0 = null
```

3.10 Implementasi Arrow Function dan Callback (02-arrow-callback.js)

Pada bagian ini dilakukan implementasi *arrow function* dan *callback*. Arrow function merupakan penulisan fungsi yang lebih ringkas, sedangkan callback adalah fungsi yang dipanggil di dalam fungsi lain.

3. Penulisan kode Latihan [Arrow Function dan Callback](#)

```
console.log("\n===LATIHAN 2===");  
  
const keHurufBesar = (str) => str.toUpperCase();  
const tambahSeru = (str) => `${str}!!!`;  
const hitungKata = (str) => str.split(' ').length;  
  
function prosesKalimat(kalimat, transformasiCallback) {  
  const hasil = transformasiCallback(kalimat);  
  console.log(hasil);  
}  
  
const kalimatUji = 'belajar javascript itu menyenangkan';  
  
console.log("--- Hasil Pengujian ---");  
prosesKalimat(kalimatUji, keHurufBesar);  
prosesKalimat(kalimatUji, tambahSeru);  
prosesKalimat(kalimatUji, hitungKata);
```

4. Jalankan hasil output di Terminal

```
===LATIHAN 2===  
--- Hasil Pengujian ---  
BELAJAR JAVASCRIPT ITU MENYENANGKAN  
belajar javascript itu menyenangkan!!!  
4  
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

arrow-callback.js:39-6
arrow-callback.js:51-6
arrow-callback.js:47-6
arrow-callback.js:47-6
arrow-callback.js:47-6
arrow-callback.js:35-6

3.11 Implementasi Array Dasar (03-array-dasar.js)

Pada bagian ini dilakukan implementasi array dasar. Array digunakan untuk menyimpan banyak data dalam satu variabel variabel.

1. Penulisan kode Latihan Array Dasar

```
console.log("\n===LATIHAN 3===");
let daftarBelanja = ['Beras', 'Gula', 'Minyak Goreng', 'Telur', 'Garam'];

console.log("--- Daftar Belanja Awal ---");
for (let i = 0; i < daftarBelanja.length; i++) {
  console.log(`${i + 1}. ${daftarBelanja[i]}`);
}

daftarBelanja.push('Susu', 'Kopi');
console.log("\n(Ditambahkan Susu dan Kopi ke akhir daftar)");

const itemDihapus = daftarBelanja.shift();
console.log(`(Menghapus item pertama: ${itemDihapus})`);

const adaSusu = daftarBelanja.includes('Susu');
console.log("\n--- Cek Ketersediaan ---");
if (adaSusu) {
  console.log("Status: Item 'Susu' tersedia di dalam daftar belanja.");
} else {
  console.log("Status: Item 'Susu' tidak ditemukan.");
}

console.log(`\nJumlah total item sekarang: ${daftarBelanja.length} item.`);
console.log("Isi daftar sekarang:", daftarBelanja);
```

2. Jalankan hasil output di Terminal

```
===LATIHAN 3===
--- Daftar Belanja Awal ---
1. Beras
2. Gula
3. Minyak Goreng
4. Telur
5. Garam

(Ditambahkan Susu dan Kopi ke akhir daftar)
(Menghapus item pertama: Beras)

--- Cek Ketersediaan ---
Status: Item 'Susu' tersedia di dalam daftar belanja.

Jumlah total item sekarang: 6 item.
Isi daftar sekarang: [ 'Gula', 'Minyak Goreng', 'Telur', 'Garam', 'Susu', 'Kopi' ]
```

3.12 Implementasi Method Array (04-array-method.js)

Pada bagian ini dilakukan implementasi method pada array. Method array digunakan untuk mengelola data seperti menambah, menghapus, atau mengubah isi array.

1. Penulisan kode Latihan Method Array

```
console.log("\n===LATIHAN 4===");
const produk = [
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Mouse', harga: 150000, stok: 0 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Monitor', harga: 3200000, stok: 0 },
  { nama: 'Headset', harga: 350000, stok: 8 }
];

const produkTersedia = produk.filter(p => p.stok > 0);
console.log("--- Produk Tersedia ---");
console.log(produkTersedia);

const namaProdukTersedia = produkTersedia.map(p => p.nama);
console.log("\n--- Nama Produk Tersedia ---");
console.log(namaProdukTersedia);

const totalInventaris = produkTersedia.reduce((total, p) => {
  return total + (p.harga * p.stok);
}, 0);
console.log(`\nTotal Nilai Inventaris: Rp${totalInventaris.toLocaleString('id-ID')}`);

console.log("\n--- Status Inventaris Lengkap ---");
produk.forEach(p => {
  const status = p.stok > 0 ? "[TERSEDIA]" : "[HABIS]";
  console.log(`${status} ${p.nama} - Rp${p.harga.toLocaleString('id-ID')} (${p.stok} unit)`);
});
```

2. Jalankan hasil output di Terminal

```
===LATIHAN 4===
--- Produk Tersedia ---
> (3) [{"-"}, {"-"}, {"-"}]

--- Nama Produk Tersedia ---
> (3) ['Laptop', 'Keyboard', 'Headset']

Total Nilai Inventaris: Rp50.700.000

--- Status Inventaris Lengkap ---
[TERSEDIA] Laptop - Rp8.500.000 (5 unit)
[HABIS] Mouse - Rp150.000 (0 unit)
[TERSEDIA] Keyboard - Rp450.000 (12 unit)
[HABIS] Monitor - Rp3.200.000 (0 unit)
[TERSEDIA] Headset - Rp350.000 (8 unit)
```

3.13 Pembuatan folder Tugas

1. Membuat folder tugas dalam folder praktikum-3 dengan mengetik “*mkdir tugas*” di Git Bash

```
acer@DESKTOP-NJEJ83T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
$ mkdir tugas
```

2. Membuat File di Dalam Folder (touch):

Selanjutnya buat dua file JavaScript baru sekaligus di dalam folder tugas menggunakan perintah touch. File tersebut diberi nama tugas-1.js tugas/tugas-2.js

```
acer@DESKTOP-NJEJ83T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma/praktikum-3 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```


3. Tampilan Isi Folder

This PC > DATA (D:) > Struktur data dan algoritma > 2025573010009_strukturdatadanalgoritma > praktikum-3 > tugas

Name	Date modified	Type	Size
 tugas-1	14/04/2026 16:15	JavaScript Source ...	0 KB
 tugas-2	14/04/2026 16:15	JavaScript Source ...	0 KB

3.14 Implementasi Tugas 1 : Sistem Nilai Mahasiswa

Bagian ini memuat kode program JavaScript untuk mengelola data nilai mahasiswa menggunakan metode array.

1. Penulisan Kode Program

```
tugas-1.js X
tugas > tugas-1.js > ...
1
2  const dataMahasiswa = [
3    { nama: 'Dewi', nilai: 85 },
4    { nama: 'Almas', nilai: 92 },
5    { nama: 'Budi', nilai: 65 },
6    { nama: 'Citra', nilai: 78 },
7    { nama: 'Dedi', nilai: 45 },
8    { nama: 'Eka', nilai: 55 }
9  ];
10
11 function hitungStatistik(arrMahasiswa) {
12   const totalMhs = arrMahasiswa.length;
13
14   const stats = arrMahasiswa.reduce((acc, mhs) => {
15     return {
16       totalNilai: acc.totalNilai + mhs.nilai,
17       tertinggi: mhs.nilai > acc.tertinggi ? mhs.nilai : acc.tertinggi,
18       terendah: mhs.nilai < acc.terendah ? mhs.nilai : acc.terendah
19     };
20   }, { totalNilai: 0, tertinggi: -Infinity, terendah: Infinity });
21
22   return {
23     total: totalMhs,
24     rataRata: (stats.totalNilai / totalMhs).toFixed(2),
25     tertinggi: stats.tertinggi,
26     terendah: stats.terendah
27   };
28 }
29
30 function filterLulus(arrMahasiswa, batasLulus) {
31   return arrMahasiswa.filter(mhs => mhs.nilai >= batasLulus);
32 }
33
34 function tambahGrade(arrMahasiswa) {
35   return arrMahasiswa.map(mhs => {
36     let grade;
37     if (mhs.nilai >= 85) grade = 'A';
38     else if (mhs.nilai >= 75) grade = 'B';
39     else if (mhs.nilai >= 60) grade = 'C';
40     else if (mhs.nilai >= 45) grade = 'D';
41     else grade = 'E';
42
43     return { ...mhs, grade: grade };
44   });
45 }
46
47 const statistik = hitungStatistik(dataMahasiswa);
48 const mahasiswaLulus = filterLulus(dataMahasiswa, 60);
49 const dataDenganGrade = tambahGrade(dataMahasiswa);
50
51 console.log("=== LAPORAN SISTEM NILAI MAHASISMA ===");
52 console.log("Total Mahasiswa : ${statistik.total}");
53 console.log("Rata-rata Nilai : ${statistik.rataRata}");
54 console.log("Nilai Tertinggi : ${statistik.tertinggi}");
55 console.log("Nilai Terendah : ${statistik.terendah}");
56
57 console.log("\n--- Daftar Mahasiswa & Grade ---");
58 dataDenganGrade.forEach((mhs, index) => {
59   console.log(`${index + 1}. ${mhs.nama} - Nilai: ${mhs.nilai} (Grade: ${mhs.grade})`);
60 });
61
62 console.log("\n--- Mahasiswa yang Lulus (Nilai >= 60) ---");
63 mahasiswaLulus.forEach(mhs => {
64   console.log(`${mhs.nama} (LULUS)`);
65 });
```

2. Eksekusi dan Verifikasi Output

File dijalankan melalui terminal VS Code dengan perintah node tugas/tugas-1.js.
Terminal

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-3> node tugas/tugas-1.js
=== LAPORAN SISTEM NILAI MAHASISWA ===
Total Mahasiswa : 6
Rata-rata Nilai : 70.00
Nilai Tertinggi : 92
Nilai Terendah : 45

--- Daftar Mahasiswa & Grade ---
1. Dewi - Nilai: 85 (Grade: A)
2. Almas - Nilai: 92 (Grade: A)
3. Budi - Nilai: 65 (Grade: C)
4. Citra - Nilai: 78 (Grade: B)
5. Dedi - Nilai: 45 (Grade: D)
6. Eka - Nilai: 55 (Grade: D)

--- Mahasiswa yang Lulus (Nilai >= 60) ---
- Dewi (LULUS)
- Almas (LULUS)
- Budi (LULUS)
- Citra (LULUS)
```

3.15 Implementasi Tugas 2 : Pangkat dan Palindrom Rekursif

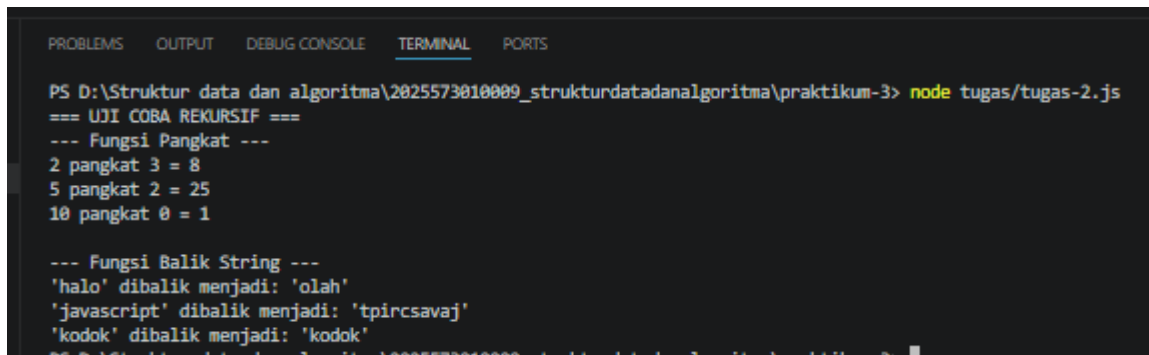
Implementasi kode pada file tugas-2.js berikut ini berfokus pada penggunaan fungsi rekursif. Fungsi ini dirancang untuk melakukan operasi matematika pangkat tanpa operator bawaan serta melakukan manipulasi string melalui pemanggilan fungsi itu sendiri secara berulang."

1. Penulisan Kode Program

```
JS tugas-2.js x
tugas > JS tugas-2.js > ...
1
2 function pangkat(basis, eksponen) {
3
4     if (eksponen === 0) {
5         return 1;
6     }
7
8     return basis * pangkat(basis, eksponen - 1);
9 }
10
11 function balikString(str) {
12     if (str.length <= 1) {
13         return str;
14     }
15
16     let karakterTerakhir = str[str.length - 1];
17     let sisaString = str.slice(0, str.length - 1);
18
19     return karakterTerakhir + balikString(sisaString);
20 }
21
22 console.log("=== UJI COBA REKURSIF ===");
23
24 console.log("---- Fungsi Pangkat ----");
25 console.log(`2 pangkat 3 = ${pangkat(2, 3)}`);
26 console.log(`5 pangkat 2 = ${pangkat(5, 2)}`);
27 console.log(`10 pangkat 0 = ${pangkat(10, 0)}`);
28
29 console.log("\n--- Fungsi Balik String ---");
30 console.log(`'halo' dibalik menjadi: '${balikString('halo')}'`); // Hasil: olah
31 console.log(`'javascript' dibalik menjadi: '${balikString('javascript')}'`);
32 console.log(`'kodok' dibalik menjadi: '${balikString('kodok')}'`);
```

2. Eksekusi dan Verifikasi Output

File dijalankan melalui terminal VS Code dengan perintah `node tugas/tugas-2.js`.
Terminal.



```
PS D:\Struktur data dan algoritma\2025573010009_strukturdatadanalgoritma\praktikum-3> node tugas/tugas-2.js
=== UJI COBA REKURSIF ===
--- Fungsi Pangkat ---
2 pangkat 3 = 8
5 pangkat 2 = 25
10 pangkat 0 = 1

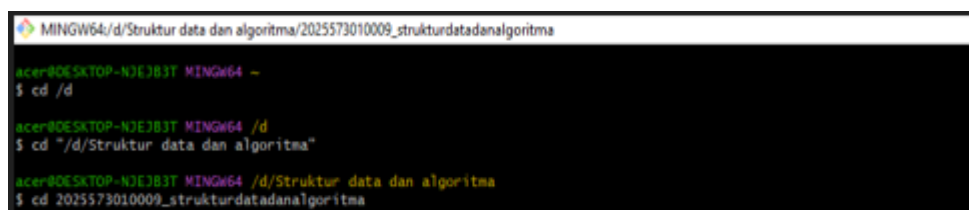
--- Fungsi Balik String ---
'halo' dibalik menjadi: 'olah'
'javascript' dibalik menjadi: 'tpircsavaj'
'kodok' dibalik menjadi: 'kodok'
```

3.16 Langkah-Langkah Submit Tugas ke GitHub

Berikut adalah langkah-langkah yang dilakukan untuk mengunggah folder praktikum-3 ke repositori GitHub sesuai dengan instruksi modul:

1. Berpindah ke Direktori Tugas (CD)

Melakukan perpindahan direktori agar terminal berada di dalam folder repositori lokal.

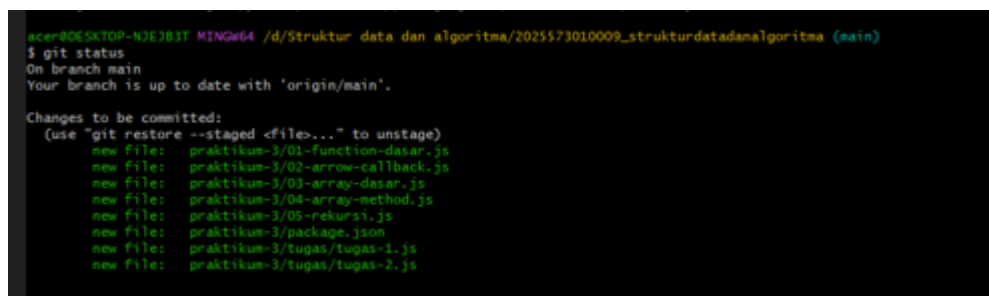


```
MINGW64/d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma
acer@DESKTOP-N3E3B3T MINGW64 ~
$ cd /d
acer@DESKTOP-N3E3B3T MINGW64 /d
$ cd "/d/Struktur data dan algoritma"
acer@DESKTOP-N3E3B3T MINGW64 /d/Struktur data dan algoritma
$ cd 2025573010009_strukturdatadanalgoritma
```

2. Pengecekan Status Awal

Menjalankan perintah untuk melihat daftar file yang baru dibuat atau diubah

Dengan mengetik “*git status*” di Git Bash



```
acer@DESKTOP-N3E3B3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   praktikum-3/01-function-dasar.js
    new file:   praktikum-3/02-arrow-callback.js
    new file:   praktikum-3/03-array-dasar.js
    new file:   praktikum-3/04-array-method.js
    new file:   praktikum-3/05-rekursi.js
    new file:   praktikum-3/package.json
    new file:   praktikum-3/tugas/tugas-1.js
    new file:   praktikum-3/tugas/tugas-2.js
```

3. Menambahkan File ke Staging Area (Git Add)

Mengumpulkan semua file praktikum ke dalam daftar tunggu kirim.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanaalgoritma (main)
$ git add praktikum-3/
```

4. Menyimpan Perubahan (Git Commit)

Membungkus perubahan dengan pesan keterangan praktikum.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanaalgoritma (main)
$ git commit -m "praktikum 3: Function & Array - arrow function,callback,iterasi array,rekursi"
[main 3d96c36] praktikum 3: Function & Array - arrow function,callback,iterasi array,rekursi
 8 files changed, 437 insertions(+)
 create mode 100644 praktikum-3/01-function-dasar.js
 create mode 100644 praktikum-3/02-arrow-callback.js
 create mode 100644 praktikum-3/03-array-dasar.js
 create mode 100644 praktikum-3/04-array-method.js
 create mode 100644 praktikum-3/05-rekursi.js
 create mode 100644 praktikum-3/package.json
 create mode 100644 praktikum-3/tugas/tugas-1.js
 create mode 100644 praktikum-3/tugas/tugas-2.js
```

5. Mengunggah ke GitHub (Git Push)

Mengirimkan hasil kerja ke server GitHub agar bisa diakses oleh dosen.

```
acer@DESKTOP-NJEJB3T MINGW64 /d/Struktur data dan algoritma/2025573010009_strukturdatadanaalgoritma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.22 KiB | 1.04 MiB/s, done.
Total 12 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/dewialmas/2025573010009_strukturdatadanaalgoritma.git
 49ac429..3d96c36  main -> main
```

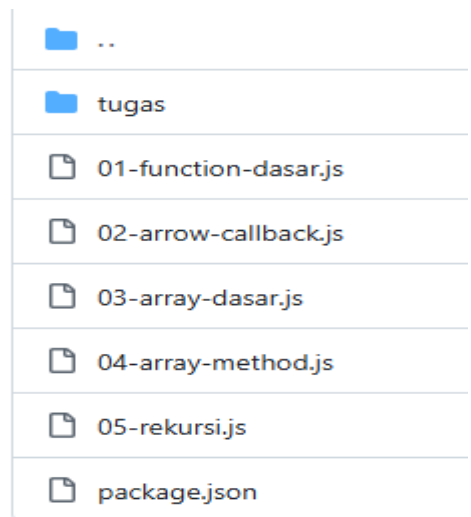
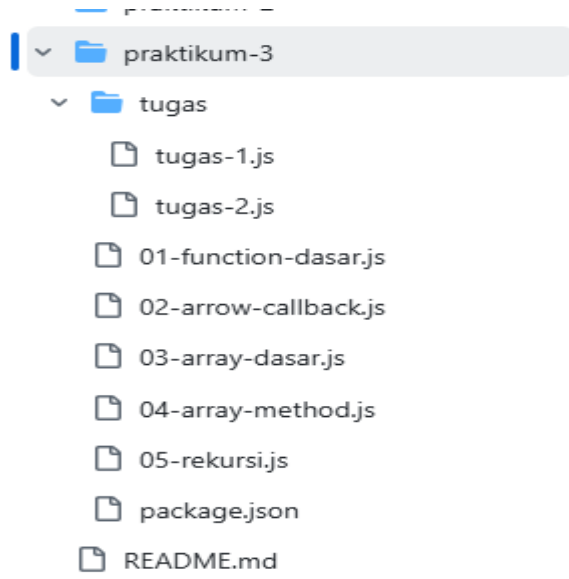
6. Verifikasi Hasil Unggahan di GitHub

Setelah muncul keterangan "*main -> main*" di Git Bash (seperti pada gambar di atas), segera membuka dashboard GitHub melalui browser. Folder praktikum-3 sudah otomatis muncul di halaman utama repositori beserta pesan commit yang sesuai.

 praktikum-3	Update tugas-1.js	38 minutes ago
---	-------------------	----------------

7. Pemeriksaan Akhir Folder dan File

Untuk memastikan seluruh file di dalam folder praktikum-3, termasuk file tugas mandiri (tugas-1.js dan tugas-2.js), telah terunggah dengan sempurna. Hal ini menandakan bahwa proses sinkronisasi dari repositori lokal ke repositori publik telah berhasil dilakukan secara menyeluruh.



BAB VI

ANALISIS

4.1 Analisis Function Dasar

Pada bagian ini, dapat dianalisis bahwa penggunaan function dasar sangat membantu dalam menyusun kode program agar lebih terstruktur. Function memungkinkan penulisan kode yang dapat digunakan kembali sehingga lebih efisien dan tidak terjadi pengulangan kode. Selain itu, function juga mempermudah dalam memahami alur program karena setiap fungsi memiliki tugas tertentu.

4.2 Analisis Arrow Function

Arrow function merupakan bentuk penulisan fungsi yang lebih ringkas dibandingkan dengan function biasa. Dari praktikum yang dilakukan, terlihat bahwa arrow function dapat mempercepat penulisan kode dan membuatnya lebih sederhana. Namun, penggunaan arrow function memerlukan ketelitian dalam penulisan karena kesalahan kecil dapat menyebabkan error pada program.

4.3 Analisis Callback

Callback merupakan konsep di mana sebuah fungsi digunakan sebagai parameter dalam fungsi lain. Berdasarkan praktikum, callback memberikan fleksibilitas dalam pemrograman karena memungkinkan suatu fungsi dijalankan di dalam fungsi lain. Hal ini membuat program menjadi lebih dinamis. Namun, pemahaman yang kurang terhadap alur callback dapat menyebabkan kebingungan dalam membaca kode.

4.4 Analisis Array Dasar

Array digunakan untuk menyimpan banyak data dalam satu variabel. Dari hasil praktikum, dapat dianalisis bahwa array sangat efektif dalam mengelola data karena setiap elemen memiliki indeks yang memudahkan dalam pengaksesan data. Hal ini membuat program menjadi lebih terorganisir dibandingkan menggunakan banyak variabel terpisah.

4.5 Analisis Method Array

Method array memberikan kemudahan dalam manipulasi data, seperti menambah, menghapus, dan mengubah isi array. Berdasarkan praktikum, penggunaan

method array sangat membantu dalam pengolahan data secara efisien. Hal ini menunjukkan bahwa JavaScript menyediakan berbagai fungsi bawaan yang mempermudah pekerjaan programmer.

4.6 Analisis Rekursi

Rekursi merupakan teknik pemrograman di mana sebuah fungsi memanggil dirinya sendiri. Dari praktikum yang dilakukan, dapat dianalisis bahwa rekursi sangat berguna dalam menyelesaikan permasalahan yang bersifat berulang, seperti perhitungan matematika. Namun, penggunaan rekursi harus memperhatikan kondisi berhenti (*base case*) agar tidak terjadi perulangan tanpa batas yang dapat menyebabkan error.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penggunaan *function*, *arrow function*, *array*, serta *Git Bash* sangat penting dalam pemrograman JavaScript.

Function membantu dalam membuat kode lebih terstruktur dan efisien. Arrow function memberikan penulisan yang lebih ringkas. Array mempermudah dalam pengelolaan data, sedangkan method array membantu dalam manipulasi data.

Selain itu, *Git Bash* sangat membantu dalam pengelolaan file dan folder sehingga proyek dapat tersusun dengan rapi dan terorganisir.

5.2 Saran

Adapun saran yang dapat diberikan adalah:

1. Mahasiswa diharapkan lebih teliti dalam penulisan kode untuk menghindari error.
2. Perlu dilakukan latihan secara rutin agar lebih memahami konsep pemrograman.
3. Disarankan untuk membiasakan penggunaan *Git* dalam setiap pengerjaan proyek.